

PROJECT EXPLAINER

High-Performance, Scalable Java Microservices Platform

Core Java 17 | Spring Boot | Asynchronous & Non-blocking I/O | Multi-threading
99.9% uptime - 1,000+ daily users - low-latency distributed system

What this document is

This document takes one line from a resume and explains the whole system behind it in everyday language, with no assumed background.

It covers: a plain-English summary, every component explained end to end, the complete request workflow step by step, and two labelled diagrams.

Anywhere a technical word appears for the first time, it is explained right there. A full glossary is at the end.

Contents

#	Section	What you will get from it
1	The project in plain English	A one-paragraph summary and a real-world analogy
2	The resume line, decoded	Every phrase translated into simple language
3	Architecture diagram	The full picture, plus how to read it layer by layer
4	Every component, end to end	What each part is, why it exists, how it is built
5	The engine room	Async, non-blocking I/O and multi-threading made simple
6	The workflow, step by step	A single request followed from click to confirmation
7	How 99.9% uptime is achieved	The specific techniques that keep it running
8	Scale and performance in numbers	What 1,000+ daily users really means
9	Glossary	Every technical term in one place
10	Likely interview questions	Questions and short, confident answers

1. The Project in Plain English

The one-paragraph summary

This project is the back end of an online application - the part users never see, but that does all the actual work. Instead of building it as one giant program, it was built as a set of small, independent programs called microservices. Each microservice owns one job: one handles user accounts, another handles orders, another handles payments, another sends notifications. They are written in Java 17 using Spring Boot, a framework that removes most of the boilerplate setup work. The services talk to each other over the network, and they are deliberately designed so that nobody ever sits around waiting. When a service needs something slow - a database read, a payment provider, an email - it hands the job off and immediately gets on with other requests. That single design choice is what keeps responses fast (low latency) and lets the system serve over a thousand users a day while staying available 99.9% of the time, which works out to less than about 45 minutes of downtime in a whole month.

The restaurant analogy

If the whole system were a restaurant:

- **The customers** are the users on a website or phone app.
- **The host at the door** is the API Gateway. Every customer goes through them, they check your reservation (your login token), and they decide which section you are seated in.
- **The specialist chefs** are the microservices. One does only grills, one does only desserts. If the dessert chef is off sick, the grill still works - the restaurant does not close.
- **The order tickets on the rail** are the message queue (Kafka). A waiter clips up a ticket and walks away immediately instead of standing at the pass waiting for the food.
- **Each chef's own fridge** is that service's own private database. No chef reaches into another chef's fridge; they ask.
- **The manager watching the floor** is the monitoring and alerting stack. They see a queue building up before customers start complaining.

The single most important idea

A slow restaurant is not usually slow because the chefs are slow. It is slow because someone is standing still, waiting.

This entire project is built around removing waiting. That is what 'asynchronous' and 'non-blocking' mean, and everything else follows from it.

2. The Resume Line, Decoded

Each phrase in the original line means something specific. Here is the plain translation of every one.

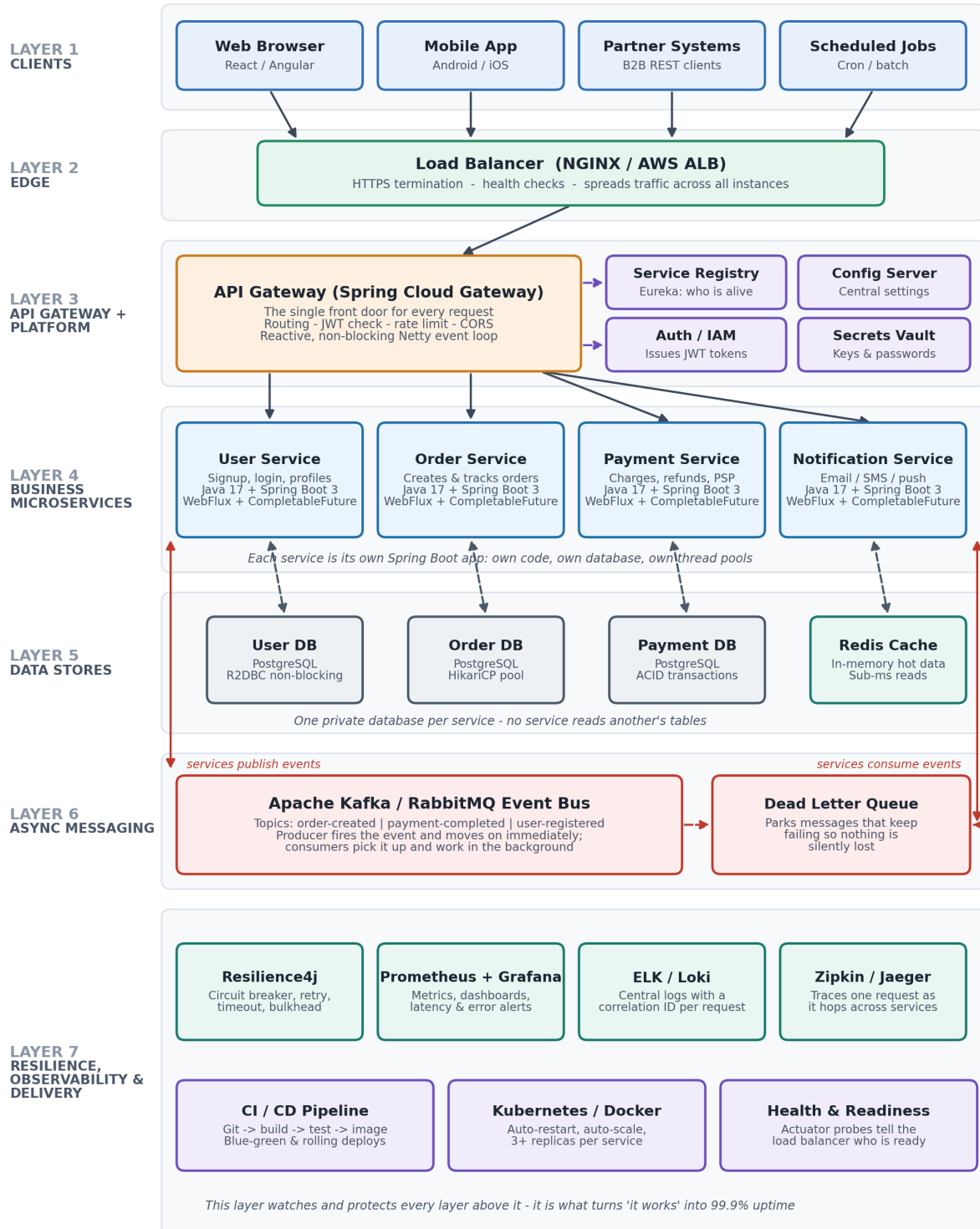
The phrase	What it actually means	Why an interviewer cares
Server-side	Code that runs on machines you control, not in the user's browser. The user never sees it.	It is where the real logic, data and security live.
Microservices	The app is split into several small independent programs instead of one big one.	Each can be updated, restarted and scaled on its own.
Core Java 17	Plain Java (no framework), version 17 - a long-term-support release with records, sealed classes, switch expressions and a better garbage collector.	Shows you use a modern, supported version, not Java 8.
Spring Boot	The framework that wires the application together and gives you a ready-to-run web server, configuration and database plumbing.	It is the industry default for Java back ends.
High-performance	It does a lot of work per second without needing more hardware.	Directly translates to lower cloud cost.
Scalable	When traffic grows you can add more copies of a service and it keeps up.	Proves the design will survive growth.
Extensible	New features can be added without rewriting what already exists.	Shows the code has clean boundaries.
99.9% uptime	Available 999 minutes out of every 1,000. Roughly 43 minutes of downtime per month is the budget.	This is a measurable promise, not a feeling.
1K+ daily users	More than a thousand distinct people use it every day.	Real production traffic, not a toy project.
Asynchronous programming	Start a slow job, do not wait for it, get told later when it is done.	The core technique behind the speed.
Non-blocking I/O	A thread never freezes while waiting for the network or disk; it goes and serves someone else.	It is what lets a few threads serve many users.
Multi-threading	Many tasks run at the same time inside one program, using thread pools you control.	Shows you understand concurrency, not just syntax.
Low-latency	The gap between the click and the answer is small - typically well under a second.	Latency is what users actually feel.
Distributed system	The program runs across several machines that talk over a network.	Networks fail, so this implies you handled failure.

3. The Architecture Diagram

The diagram below is the whole system on one page. It is organised in seven horizontal layers. Read it top to bottom: a request enters at the top, gets checked and routed in the middle, is stored near the bottom, and the very bottom layer is what keeps the whole thing alive.

High-Performance Java 17 / Spring Boot Microservices Platform

Asynchronous | Non-blocking I/O | Multi-threaded | 99.9% uptime for 1,000+ daily users



Read top to bottom: a request enters at Layer 1, is routed at Layer 3, handled at Layer 4, stored at Layer 5, echoed as events at Layer 6, and kept healthy by Layer 7.

Figure 1 - Full system architecture. Arrows show the direction information travels. Dashed arrows are supporting lookups; red arrows are events.

How to read the diagram, layer by layer

Layer	What sits there	What it does, in one sentence
Layer 1 - Clients	Browser, mobile app, partner systems, scheduled jobs	Anything that can send a request into the system; note that not all traffic is human.
Layer 2 - Edge	Load balancer	The single public address; it spreads incoming traffic across all healthy copies of the system.
Layer 3 - Gateway & platform	API Gateway, Service Registry, Config Server, Auth, Secrets Vault	The front door plus the support services that tell the front door where to go and who is allowed in.
Layer 4 - Microservices	User, Order, Payment, Notification services	The actual business logic; each box is a separate Spring Boot application with its own team, code and release cycle.
Layer 5 - Data	One PostgreSQL database per service, plus a shared Redis cache	Where information is kept permanently, and where hot information is kept temporarily for speed.
Layer 6 - Async messaging	Kafka / RabbitMQ event bus and a Dead Letter Queue	The 'do not wait' backbone; services announce that something happened and move on immediately.
Layer 7 - Resilience & operations	Resilience4j, Prometheus, Grafana, ELK, Zipkin, CI/CD, Kubernetes	The safety net and the eyes; this layer is what turns 'it works on my machine' into 99.9% uptime.

Three rules the diagram is enforcing

1. Everything enters through one door. No client ever calls a microservice directly. This means authentication, rate limiting and logging are done once, in one place, instead of being repeated (and forgotten) in every service.
2. Every service owns its own data. Notice there is no shared database in the diagram. If the Order Service needs user details it asks the User Service over the network - it never reaches into the user tables itself. This is what makes services genuinely independent.
3. Arrows going down are commands; arrows going sideways into Kafka are announcements. A command says 'do this and tell me the result'. An announcement says 'this happened, whoever cares can react'. Announcements are what let the system stay fast.

4. Every Component, Explained End to End

Each component below is described the same way: what it is, why it exists, how it is actually built, and what would break without it.

4.1 The edge: getting traffic in safely

Load Balancer (NGINX or AWS ALB)

What it is: A single public address that sits in front of everything and forwards each incoming request to one of several identical copies of the system.

Why it exists: One server can only handle so much, and one server can crash. The load balancer lets you run three or more copies and treat them as one, so a crash becomes invisible to users.

How it is built: It performs HTTPS termination (decrypting the secure connection once, at the edge), then picks a target instance using a strategy such as round-robin or least-connections. It calls each instance's health endpoint every few seconds and silently removes any instance that stops answering.

Remove it and: You are back to one server. Any restart, deploy or crash becomes visible downtime, and the 99.9% target is impossible.

API Gateway (Spring Cloud Gateway)

What it is: The single front door for the application. Every request from every client passes through it before reaching any business logic.

Why it exists: Without it, each microservice would have to implement login checks, rate limits, CORS rules and logging separately - which means four chances to get security wrong instead of one.

How it is built: It is itself a Spring Boot application, built on Spring Cloud Gateway, which runs on Netty and is reactive and non-blocking by default. It matches the incoming URL against a route table, validates the JWT token, applies a rate limit per user, adds a correlation ID to the request headers, then forwards the call. Because it is non-blocking, a handful of threads can hold thousands of in-flight requests at once.

Remove it and: Clients would need to know the address of every service, security would be inconsistent, and you could not change internal service layout without breaking clients.

Service Registry (Netflix Eureka)

What it is: A live phone book of which service instances are currently running and healthy.

Why it exists: In a cloud environment, instances are created and destroyed constantly and their IP addresses change. Hard-coding addresses in configuration files stops working almost immediately.

How it is built: Every service registers itself on startup and then sends a heartbeat every 30 seconds. If heartbeats stop, the registry removes it. When the Gateway needs the Order Service, it asks the registry for the current list of healthy instances and picks one.

Remove it and: The Gateway would send traffic to machines that no longer exist, producing connection errors during every single deployment.

Config Server

What it is: One central place that holds the settings - timeouts, feature flags, connection limits, log levels - for every service and every environment.

Why it exists: Settings baked into the application would require a rebuild and redeploy to change. In an incident, you need to raise a timeout in seconds, not in an hour.

How it is built: Services fetch their configuration on startup and can refresh it live. The configuration itself is usually stored in a Git repository, so every change to production settings is version-controlled and reviewable.

Remove it and: Configuration drifts between environments, and 'it works in test but not in production' becomes a weekly event.

Auth / IAM and the Secrets Vault

What it is: The component that verifies a username and password and issues a signed JWT token, plus a separate encrypted store for passwords, API keys and certificates.

Why it exists: Identity must be checked once and then trusted cheaply; and credentials must never sit in source code.

How it is built: On login, the Auth service returns a JWT - a small signed token containing the user id, roles and an expiry time. The client sends it on every subsequent request. The Gateway verifies the signature mathematically, without any database call, which is why authentication adds well under a millisecond. Secrets are injected at runtime from a vault, never committed to Git.

Remove it and: Either every request needs a database lookup to check identity (slow), or credentials end up in the repository (a serious security incident waiting to happen).

4.2 Inside a single microservice

Every one of the four business services has the same internal shape. Understanding one means understanding all of them. A request travels inward through these layers and the response travels back out.

Layer inside the service	Its job	Typical Java / Spring detail
Controller	Receives the HTTP request and returns the HTTP response. It contains no business rules.	@RestController with @GetMapping / @PostMapping; returns Mono or CompletableFuture for async paths.
DTO (Data Transfer Object)	The shape of the data going in and out - deliberately separate from the database shape.	Java 17 records are ideal here: one line, immutable, and safe to share across threads.
Validation	Rejects bad input immediately, before any work or database call happens.	@Valid with @NotNull, @Size, @Email annotations; failures return 400 without touching the database.
Service layer	The actual business rules - the part that is genuinely unique to your company.	@Service classes; transaction boundaries declared with @Transactional.
Repository	Reads and writes the database without hand-written SQL for common cases.	Spring Data JPA repository interfaces, or R2DBC repositories on the reactive path.
Entity	The Java object that maps onto a database table.	@Entity with @Id; kept strictly inside the service, never exposed over HTTP.
Event publisher / listener	Announces that something happened, and reacts to announcements from others.	KafkaTemplate to publish; @KafkaListener to consume.

Layer inside the service	Its job	Typical Java / Spring detail
Configuration	Thread pool sizes, timeouts, retry counts, connection pool limits.	application.yml plus @ConfigurationProperties classes, supplied by the Config Server.

Why the layers matter

The point of this separation is that business rules never depend on HTTP or on the database.

That is what makes the system extensible: you can switch from REST to messaging, or from PostgreSQL to something else, and the service layer in the middle does not change.

4.3 The four business services

User Service

What it is: Owns everything about a person's account: registration, login, profile details, roles and permissions.

Why it exists: Identity is needed by every other service, so it is isolated into one place with one owner and one database.

How it is built: Exposes REST endpoints for signup, profile read and update. Passwords are stored as BCrypt hashes, never as text. On successful registration it publishes a user-registered event so other services can react without the User Service knowing they exist.

Remove it and: Account logic would be duplicated across services and would eventually disagree with itself.

Order Service

What it is: The orchestrator. It creates orders, tracks their status through the lifecycle, and coordinates the other services.

Why it exists: Most real business processes span several services. Something has to own the overall process and know what state it is in.

How it is built: Writes the order in a PENDING state first, then publishes an order-created event and returns to the user immediately. It later listens for payment-completed and moves the order to CONFIRMED. This 'save first, announce second, confirm later' pattern is what keeps the user-facing response fast.

Remove it and: The user would wait for payment, inventory and email to all finish before seeing any response - turning a 50-millisecond reply into several seconds.

Payment Service

What it is: Handles charges, refunds and all communication with the external payment provider.

Why it exists: Payments are the highest-risk and slowest part of the system. Isolating them means their slowness and their failures cannot spread.

How it is built: Consumes order-created events from Kafka. Calls the external provider asynchronously with a strict timeout, wrapped in a Resilience4j circuit breaker. Uses an idempotency key so that a retried message can never charge a customer twice, and writes to its database inside a proper ACID transaction.

Remove it and: A slow external payment provider would hold threads open across the whole application and eventually freeze services that have nothing to do with payments.

Notification Service

What it is: Sends confirmation emails, SMS messages and push notifications.

Why it exists: Notifications are slow, they fail often, and nobody should ever be kept waiting for one.

How it is built: It is purely event-driven: it has no public API at all and only reacts to events on Kafka. Failed sends are retried with exponential backoff, and messages that keep failing land in the Dead Letter Queue for later inspection.

Remove it and: Email delays would show up as slow page loads, and a failing email provider would look to users like the whole site was broken.

4.4 The data layer

PostgreSQL - one private database per service

What it is: A relational database, with a separate one for each service. The Order Service physically cannot query the user tables.

Why it exists: Shared databases are the most common reason microservices fail in practice: every service ends up coupled to every other service's table structure, and nobody can change anything.

How it is built: Each service owns its own schema and its own migration history, managed by a tool such as Flyway or Liquibase so schema changes are versioned alongside code. Cross-service data is obtained by API call or by listening to events.

Remove it and: You get a 'distributed monolith' - all the operational cost of microservices with none of the independence.

Connection pooling - HikariCP and R2DBC

What it is: A pool of pre-opened database connections that are borrowed and returned, rather than opened fresh each time.

Why it exists: Opening a database connection takes tens of milliseconds. Doing that per request would dominate your response time.

How it is built: HikariCP is the default pool in Spring Boot for standard (blocking) JDBC access. On the reactive path, R2DBC provides genuinely non-blocking database access, so a thread is not held hostage while the database thinks. Pool size is tuned deliberately - a common mistake is making it far too large, which just moves the queue from your application into the database.

Remove it and: Under load the application spends most of its time opening and closing connections, and the database runs out of them.

Redis cache

What it is: An in-memory key-value store holding frequently requested data such as product details, user profiles and computed results.

Why it exists: Memory is roughly a thousand times faster than disk. Most systems read the same small set of data over and over.

How it is built: The service checks Redis first (a cache hit returns in under a millisecond); on a miss it reads the database and writes the result back into Redis with a time-to-live so stale data expires automatically. Redis is also used for distributed rate limiting and for locks that must be shared across instances.

Remove it and: Every read hits PostgreSQL, response times climb, and the database becomes the bottleneck long before the application does.

4.5 The asynchronous backbone

Kafka / RabbitMQ event bus

What it is: A durable pipeline of messages. Services publish events onto named topics; other services subscribe to the topics they care about.

Why it exists: It is what decouples the services. The Order Service does not know that the Notification Service exists, and does not need to.

How it is built: Events are published to topics such as order-created, payment-completed and user-registered. Kafka stores each event on disk, so a consumer that was offline can catch up on restart - nothing is lost. Multiple consumers can read the same event independently, so adding a new reaction to an existing event requires no change to the publisher at all. This is exactly what 'extensible' means in practice.

Remove it and: Services would have to call each other directly and synchronously, so one slow service would make every service slow, and any service being down would fail the whole request.

Dead Letter Queue (DLQ)

What it is: A holding area for messages that have failed repeatedly.

Why it exists: Some failures are permanent - malformed data, a deleted record. Retrying those forever would block every message behind them.

How it is built: After a configured number of failed attempts, the message is moved to the DLQ with its error details attached. An alert fires, an engineer inspects it, and the message can be fixed and replayed once the underlying problem is resolved.

Remove it and: One poisonous message blocks its partition indefinitely, silently stopping all processing behind it.

4.6 Resilience, observability and delivery

Resilience4j - circuit breaker, retry, timeout, bulkhead

What it is: A small library that wraps calls to anything that might be slow or fail, and controls what happens when it does.

Why it exists: In a distributed system, the default behaviour of a failure is to spread. One slow dependency ties up threads, which makes the caller slow, which ties up its caller's threads. This is a cascading failure, and it is how whole platforms go down.

How it is built: The circuit breaker watches the failure rate; once it crosses a threshold it 'opens' and fails instantly for a cooling-off period instead of waiting on a dependency that is clearly broken, then cautiously tries again. Timeouts put a hard ceiling on any wait. Retries handle brief network blips, with exponential backoff and jitter so that everyone does not retry in unison. Bulkheads limit how many threads any single dependency can consume, so payments cannot starve logins.

Remove it and: One slow third party takes the entire platform down with it - and this is by far the most common cause of real outages.

Metrics: Prometheus and Grafana

What it is: Numbers collected continuously - requests per second, error rate, response time percentiles, thread pool usage, queue depth - and dashboards that show them.

Why it exists: You cannot promise 99.9% uptime if you cannot measure whether you are meeting it.

How it is built: Spring Boot Actuator exposes a metrics endpoint; Prometheus scrapes it every few seconds and stores the history. Grafana draws the dashboards, and alerts fire when a threshold is crossed. The key measure is the 95th and 99th percentile latency, not the average - averages conveniently hide the slow requests that users actually complain about.

Remove it and: You discover outages from user complaints, which means your response time is measured in hours rather than minutes.

Centralised logs and distributed tracing (ELK, Zipkin)

What it is: All logs from all services collected in one searchable place, plus traces that follow a single request across every service it touched.

Why it exists: When a request crosses five services, reading five separate log files and trying to line them up by timestamp is nearly impossible.

How it is built: A unique correlation ID is generated at the Gateway and attached to the request; every service includes it in every log line. Zipkin or Jaeger then draws the request as a timeline of spans, showing exactly which hop consumed the time. Debugging goes from an afternoon of guessing to a two-minute lookup.

Remove it and: Mean time to resolve an incident increases dramatically, which directly eats the uptime budget.

Kubernetes / Docker and health probes

What it is: Containers package each service with everything it needs to run; Kubernetes runs them, keeps the right number alive, and restarts anything that dies.

Why it exists: Machines fail, processes leak memory, and traffic varies through the day. Recovery has to be automatic, because a human cannot respond in seconds at three in the morning.

How it is built: Each service runs as three or more replicas spread across machines. Spring Boot Actuator exposes liveness and readiness probes: liveness answers 'am I alive, or should I be restarted?', readiness answers 'am I ready for traffic yet?'. Readiness is what prevents a service that is still warming up from receiving requests. Horizontal autoscaling adds replicas when CPU or request rate rises.

Remove it and: Every failure needs a human, and recovery takes minutes or hours instead of seconds.

CI/CD pipeline

What it is: Automation that takes committed code through build, test, security scan, container image and deployment.

Why it exists: Most outages are caused by deployments. Making deployment boring and repeatable is one of the highest-value things you can do for uptime.

How it is built: On every commit: compile, run unit and integration tests, scan dependencies for known vulnerabilities, build the container image, then deploy using a rolling or blue-green strategy so old instances only retire once new ones report healthy. If error rates spike, the deployment is rolled back automatically.

Remove it and: Releases become risky manual events, so they happen rarely and in large batches - which makes each one even riskier.

5. The Engine Room: Async, Non-blocking I/O and Multi-threading

These three phrases are the technical heart of the resume line, and they are frequently confused with each other. They are related but genuinely different ideas.

The coffee shop that explains all three

Approach	In the coffee shop	In the system	Result
Blocking / synchronous	One barista takes your order, makes it, hands it over, and only then serves the next person.	A thread takes a request and stays frozen until the database replies.	Simple to reason about, but the queue grows fast.
Multi-threading	Hire ten baristas. Ten customers served at once, but each still stands and waits for the milk to steam.	A thread pool handles many requests in parallel; each thread still blocks on I/O.	Better, but threads are expensive - roughly 1 MB of memory each - so you cannot simply add thousands.
Asynchronous / non-blocking	One barista takes your order, starts the machine, immediately takes the next order, and calls out names as drinks finish.	A thread starts the database call, registers a callback, and moves on. When the answer arrives, any free thread completes the work.	A small number of threads serves a very large number of requests. This is the design used here.

How each one appears in the actual code

Multi-threading in Core Java 17

- **ExecutorService and thread pools** - you create a pool with a fixed size rather than spawning threads ad hoc. Unbounded thread creation is one of the classic ways to bring down a Java service.
- **CompletableFuture** - the standard way to express 'do this, then that, and combine the results'. Independent calls run at the same time using `allOf()`, so three 100 ms calls take 100 ms in total rather than 300 ms.
- **Thread safety** - shared state is protected using immutable objects (Java 17 records are perfect for this), concurrent collections such as `ConcurrentHashMap`, and atomic types instead of manual locks wherever possible.
- **Separate pools per concern** - database work, external HTTP calls and background jobs get their own pools, so a slow external API cannot consume the threads needed to serve normal traffic.

Non-blocking I/O

Traditional Java I/O blocks: the thread calls `read()` and simply stops until data arrives. Non-blocking I/O inverts this. The operating system is asked to notify the application when data is ready, and the thread goes off to do other work in the meantime. In this project that comes from Netty (the server underneath Spring WebFlux and Spring Cloud Gateway), from WebClient for outgoing HTTP calls, and from R2DBC for reactive database access. The practical effect: a service with only a handful of event-loop threads can hold thousands of connections open simultaneously, because a waiting connection costs almost nothing.

Asynchronous programming

- **@Async methods** - annotate a method and Spring runs it on a background pool, returning control to the caller immediately.
- **Event-driven flow** - publishing to Kafka and returning is the largest async win in the whole system; the user's response does not wait for payment, inventory or email.
- **Reactive types** - Mono (zero or one result) and Flux (many results) from Project Reactor describe a pipeline of work that has not happened yet, and only runs when subscribed to.

An honest note on versions

Java 17 is the long-term-support release this project targets, and its concurrency toolkit is thread pools, `CompletableFuture` and reactive libraries such as Project Reactor.

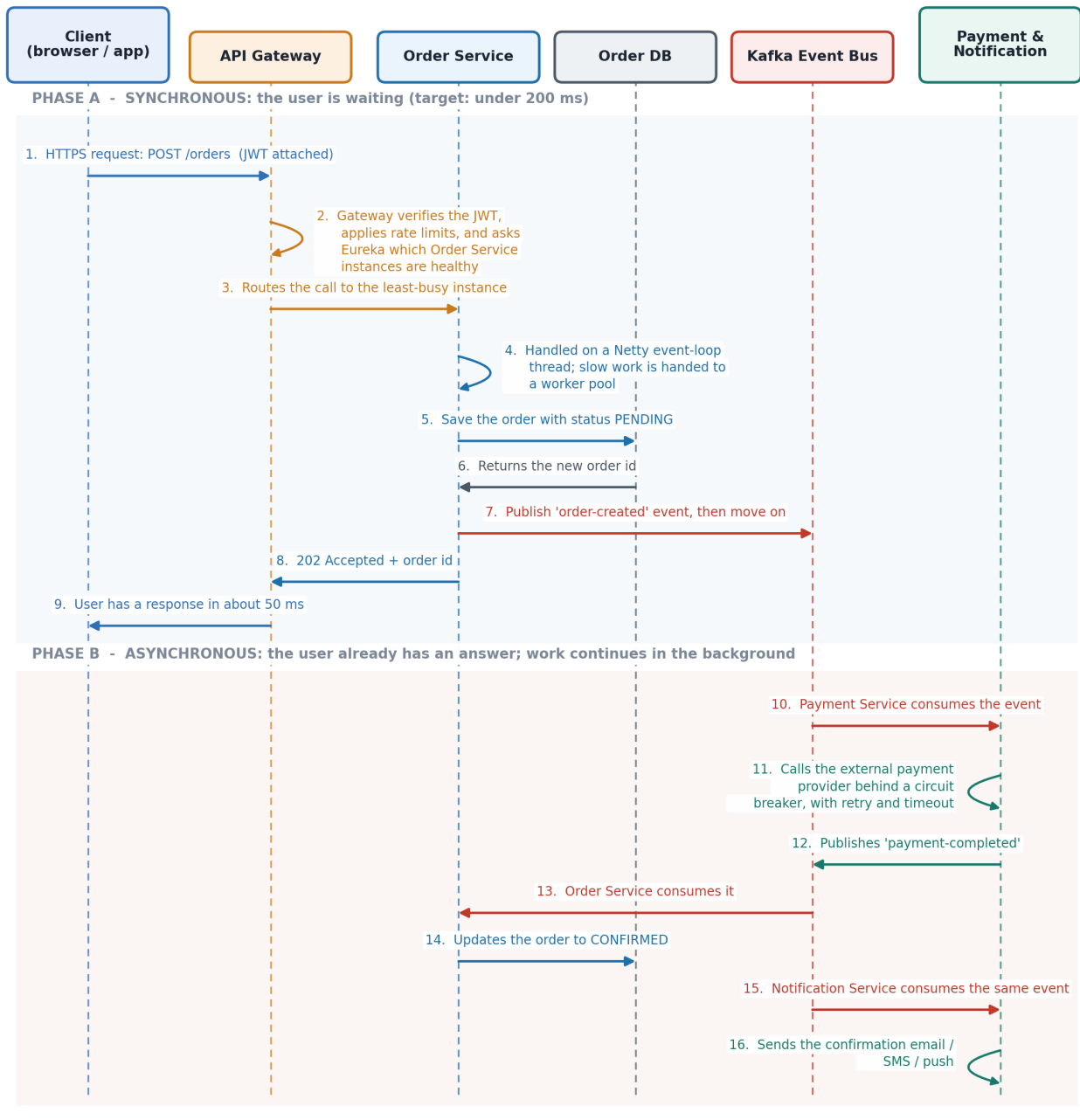
Virtual threads - which make simple blocking code behave almost as efficiently as reactive code - arrived as a finished feature in Java 21. Mentioning that you know this distinction, and that upgrading would let you simplify the reactive code, reads very well in an interview.

6. The Workflow, Step by Step

The diagram below follows one single request - a user placing an order - from the moment they click through to the confirmation email. The upper half is the part the user waits for. The lower half happens after they already have their answer.

End-to-End Workflow: what happens when a user places an order

Steps 1-9 are the fast synchronous path the user waits for. Steps 10-16 keep running in the background.



If a background step fails, Kafka retries it; after repeated failures the message is parked in the Dead Letter Queue for replay.

The user's response never waits on payment or email - that is exactly what 'asynchronous' buys you.

Figure 2 - The complete lifecycle of one request. Vertical dashed lines are components; horizontal arrows are messages between them, in time order.

Phase A - the synchronous path (the user is waiting)

Step	What happens	Why it is done this way
1	The client sends POST /orders over HTTPS with a JWT token in the Authorization header.	HTTPS protects the data in transit; the token proves identity without sending a password again.
2	The Gateway verifies the token's signature, applies the per-user rate limit, and asks the Service Registry which Order Service instances are healthy.	Signature checking is pure maths - no database call - so it costs well under a millisecond. The rate limit stops one client from consuming everything.
3	The Gateway forwards the request to the least-busy healthy instance and attaches a correlation ID.	Load is spread evenly, and the correlation ID will now appear in every log line for this request across every service.
4	The Order Service picks the request up on a Netty event-loop thread, validates the input, and hands any slow work to a worker pool.	Validation fails fast and cheaply. Keeping slow work off the event loop is the rule that makes non-blocking servers work.
5	The order is written to the Order database with status PENDING, using a pooled connection.	Recording the intent first means that even if everything downstream fails, the order is not lost.
6	The database returns the newly generated order id.	The id is what the user will use to track the order.
7	The service publishes an order-created event to Kafka and immediately moves on.	This is the pivotal moment: the service announces the fact and does not wait for anyone to react to it.
8	The service returns 202 Accepted with the order id.	202 means 'accepted for processing' rather than 'fully complete' - an honest answer, since payment has not happened yet.
9	The response travels back through the Gateway to the user in roughly 50 milliseconds.	The user has confirmation almost instantly, even though the real work has barely started.

Phase B - the asynchronous path (the user has already been answered)

Step	What happens	Why it is done this way
10	The Payment Service, subscribed to the order-created topic, consumes the event.	It works from the queue at its own pace; a traffic spike lengthens the queue rather than crashing the service.
11	It calls the external payment provider behind a circuit breaker, with a timeout and a retry policy, using an idempotency key.	The provider is the slowest and least reliable dependency. The circuit breaker stops its problems from becoming your problems; the idempotency key guarantees no double charge on retry.
12	On success it publishes a payment-completed event.	Again it announces rather than calling anyone directly, so new listeners can be added later without touching this code.
13	The Order Service consumes payment-completed.	It is listening for the outcome of the process it started.
14	The order status is updated from PENDING to CONFIRMED.	The user's tracking page now reflects reality; the state machine has advanced.

Step	What happens	Why it is done this way
15	The Notification Service consumes the same event, entirely independently.	Two services react to one event without knowing about each other. Adding a third - analytics, say - requires no change anywhere else.
16	It sends the confirmation email, SMS or push notification.	The slowest, least reliable step in the entire flow happens where no user is waiting on it.

What happens when something goes wrong

Failure	What the system does	What the user sees
A service instance crashes mid-request	The load balancer stops routing to it within seconds; Kubernetes restarts it; the client retries once.	Nothing, or a single brief retry.
The payment provider becomes slow	Timeouts fire, the circuit breaker opens, calls fail fast instead of piling up, and events wait safely in Kafka.	Orders are still accepted; confirmation simply arrives a little later.
The database is briefly unavailable	Connection pool retries with backoff; requests that cannot complete return a clear 503 rather than hanging.	A short, honest error rather than a frozen page.
A single message is malformed	It is retried a few times, then moved to the Dead Letter Queue, and an alert fires.	Nothing - every other message keeps flowing normally.
Traffic suddenly triples	Autoscaling adds replicas; the queue absorbs the burst; rate limits protect the core.	Slightly slower responses, but no errors and no outage.
A bad release goes out	Health checks fail on the new instances, the rollout halts, and traffic stays on the old version.	Nothing - the broken version never receives real traffic.

One more workflow: what happens when a service starts up

1. The container starts and the Spring Boot application begins loading.
2. It fetches its configuration from the Config Server, and its credentials from the secrets vault.
3. It opens its database connection pool and warms it with a few live connections.
4. It registers itself with the Service Registry and begins sending heartbeats.
5. Kafka consumer groups are joined and topic partitions are assigned to it.
6. The liveness probe starts passing - the process is alive.
7. Only once caches are warm and dependencies respond does the readiness probe pass.
8. The load balancer sees readiness and begins sending real traffic. Not one request was dropped during startup.

7. How 99.9% Uptime Is Actually Achieved

99.9% availability is a budget, not a slogan. It allows a specific and quite small amount of downtime, so every technique below exists to protect that budget.

Availability target	Downtime allowed per day	Per month	Per year
99% (two nines)	About 14 minutes	About 7 hours	About 3.7 days
99.9% (three nines) - this project	About 1.4 minutes	About 43 minutes	About 8.8 hours
99.99% (four nines)	About 9 seconds	About 4 minutes	About 53 minutes

The techniques, and what each one buys you

Technique	In plain terms	What it prevents
Redundancy	Run three or more copies of every service across different machines.	One machine failing takes the whole system down.
Health probes	Every service constantly answers 'am I alive?' and 'am I ready?'.	Traffic being sent to a process that cannot serve it.
Automatic restarts	A dead or hung process is replaced without any human involvement.	Overnight failures lasting until someone wakes up.
Circuit breakers	Stop calling a dependency that is clearly broken; fail fast instead.	A cascading failure spreading from one service to all of them.
Timeouts everywhere	No call is ever allowed to wait indefinitely.	Thread pools filling with stuck requests until nothing responds.
Bulkheads	Each dependency gets its own limited slice of threads.	One slow integration starving unrelated features.
Async messaging	Traffic spikes lengthen a queue instead of overwhelming a service.	Load spikes turning directly into outages.
Rolling / blue-green deploys	New instances must prove healthy before old ones retire.	Deployment windows becoming downtime windows.
Automated rollback	Error rates spiking after a release triggers an automatic revert.	A bad release staying live while people debate it.
Monitoring and alerting	Humans are paged by thresholds, not by angry users.	Long detection times quietly consuming the whole budget.
Graceful shutdown	On stop, a service finishes in-flight requests before exiting.	Errors for users who were mid-request during a deploy.
Idempotency	Doing the same operation twice has the same effect as doing it once.	Retries creating duplicate orders or duplicate charges.

8. Scale and Performance, in Numbers

It helps to be precise about what the numbers on a resume actually represent, because an interviewer may well ask you to do this arithmetic out loud.

Measure	Realistic figure	What it tells you
Daily active users	1,000+	Genuine production traffic with real users and real consequences.
Requests per user per day	Roughly 30 to 60	A typical session involves many small API calls, not one big one.
Total requests per day	Roughly 30,000 to 60,000	Steady, meaningful load.
Average requests per second	Under 1 per second	Average load is genuinely modest - and this is the honest picture.
Peak requests per second	20 to 50	Traffic is never evenly spread; peaks are what the design must survive.
Target p95 latency	Under 200 milliseconds	95 out of 100 requests finish faster than this.
Target p99 latency	Under 500 milliseconds	Even the slowest 1% stay acceptable - this is the hard one.
Threads needed with blocking I/O	Hundreds	Each one costs about 1 MB of memory just to exist.
Threads needed with non-blocking I/O	Roughly the number of CPU cores	This is the entire efficiency argument in a single row.

Be honest about the scale - it is a strength, not a weakness

1,000 daily users is not enormous traffic, and an experienced interviewer knows that. Claiming it is 'massive scale' damages your credibility.

The right framing: the interesting engineering here was not raw volume. It was keeping p99 latency low, keeping the system available through deployments and third-party failures, and building it so that a ten-fold increase in traffic needs more replicas rather than a rewrite.

That answer shows judgement, which is what the question is really testing.

9. Glossary

Term	Plain meaning
ACID	A guarantee that a database transaction either fully happens or fully does not - never halfway.
API	A defined way for one program to ask another program to do something.
Asynchronous	Start a job, do not wait for it, be told later when it is finished.
Backoff	Waiting progressively longer between retries instead of hammering a struggling system.
Blocking	A thread stops and does nothing while it waits for something slow.
Bulkhead	A limit on how many threads one dependency may consume, so it cannot starve the rest.
Circuit breaker	A switch that trips after repeated failures and fails calls instantly for a while.
Container	An application packaged with everything it needs, so it runs identically anywhere.
Correlation ID	A unique tag attached to a request so its journey can be traced across every service.
DTO	A simple object that carries data in or out, kept separate from database objects.
Event	A record that something happened, published for anyone interested to react to.
Event loop	A small number of threads that continuously handle whichever connections are ready.
Horizontal scaling	Adding more copies of a service, rather than a bigger machine.
I/O	Input and output - network calls, disk reads, database queries. The slow parts.
Idempotent	Safe to repeat: doing it twice has the same result as doing it once.
JWT	A signed token proving who you are, verifiable without a database lookup.
Latency	The time between asking and receiving an answer.
Load balancer	Spreads incoming traffic across several identical copies of a service.
Microservice	One small program doing one job, deployable independently.
Non-blocking I/O	Waiting for data without tying up a thread.
p95 / p99 latency	The time under which 95% or 99% of requests complete. Far more useful than an average.
Rate limiting	Capping how many requests one client may make in a period.
REST	A common convention for HTTP APIs using URLs and standard verbs.
Throughput	How much work the system completes per second.
Thread	A single line of execution inside a program; several can run at once.
Thread pool	A fixed set of reusable threads, so threads are not created endlessly.
Uptime	The percentage of time a system is available and working.

10. Likely Interview Questions

These are the questions this resume line invites. Short, confident answers work better than long ones.

Q. Why microservices instead of one application?

A. Independent deployment and independent failure. The Notification Service can be released ten times a day without touching payments, and if it goes down, orders still work. The trade-off is real operational complexity, which is why the monitoring and resilience layer matters as much as the services themselves.

Q. What is the difference between asynchronous and non-blocking?

A. Asynchronous is about the shape of your code - you start work and get called back later. Non-blocking is about the thread - it never sits frozen waiting on I/O. You can be asynchronous while still blocking a thread underneath. Doing both together is what actually delivers the efficiency.

Q. How did you actually achieve low latency?

A. Four things: never blocking on I/O in the request path, caching hot data in Redis so most reads never reach the database, moving everything non-essential onto Kafka so the user's response does not wait for it, and measuring p95 and p99 rather than averages so the slow requests could not hide.

Q. How do you know it was 99.9%?

A. It was measured. Prometheus scraped health and error metrics continuously, Grafana dashboards tracked availability, and alerts fired on threshold breaches. 99.9% is roughly 43 minutes of downtime a month - a number you can only defend if you were watching it.

Q. What happens if the payment provider goes down?

A. The circuit breaker opens after the failure rate crosses its threshold, so calls fail immediately instead of stacking up and consuming threads. Orders continue to be accepted because those events sit safely in Kafka. When the provider recovers, the backlog is processed. Users see delayed confirmations, not an outage.

Q. Why one database per service?

A. So services stay genuinely independent. A shared database means any schema change can break another team, and you end up with a distributed monolith - all of the operational cost, none of the independence.

Q. How do you prevent duplicate charges when a message is retried?

A. Idempotency keys. Each payment attempt carries a unique key; if the same key arrives again the provider and our own service recognise it and return the original result instead of charging a second time.

Q. What would you do differently now?

A. Move to Java 21 for virtual threads, which would let much of the reactive code be written as simple blocking code with the same efficiency and far better readability. I would also add contract testing between services, because that was the main source of integration surprises.

Q. How did multi-threading show up concretely?

A. Fixed-size thread pools sized per dependency rather than one shared pool, `CompletableFuture.allOf()` to run independent calls concurrently, and immutable objects using Java 17 records so shared state did not need locking in the first place.

Q. What was the hardest problem you solved?

A. Pick a real one and tell it as a story: the symptom, how you found it in the traces or metrics, what actually caused it, and what you changed so it could not happen again. A specific debugging story is worth more than any list of technologies.

End of document